

UNIVERSIDADE DO VALE DO ITAJAÍ – UNIVALI
ESCOLA POLITÉCNICA

DISCIPLINA: SISTEMAS OPERACIONAIS

Hugo Pinheiro Bertoglio
Fabricio Ricardo Prebianca
Joao Victor Guero

SISTEMA DE PROCESSAMENTO PARALELO DE REQUISIÇÕES A UM
BANCO DE DADOS

Avaliação M1 – IPC, Threads e Paralelismo

ITAJAÍ
2026

1 INTRODUÇÃO

Sistemas gerenciadores de banco de dados precisam atender muitas requisições ao mesmo tempo sem comprometer a integridade das informações armazenadas. Para isso, combinam dois mecanismos centrais estudados na disciplina de Sistemas Operacionais que permite que programas distintos troquem dados, e a programação concorrente com threads, que permite que um mesmo processo execute várias tarefas simultaneamente.

2 ENUNCIADO DO PROJETO

A proposta da Avaliação M1 consiste em desenvolver um “Sistema de Processamento Paralelo de Requisições a um Banco de Dados” com as seguintes características:

- um processo cliente envia requisições de consulta ou alteração ao servidor via IPC (pipe ou memória compartilhada), como “SELECT nome WHERE id=5” ou “INSERT id=7 nome='João'”;
- um processo servidor lê as requisições e utiliza um pool de threads para processá-las em paralelo;
- as threads acessam um banco de dados simulado (vetor ou arquivo) e utilizam mutex ou semáforo para garantir a integridade dos dados;
- devem ser suportadas as operações INSERT, DELETE, SELECT (por ID) e UPDATE (por ID);
- as respostas podem ser devolvidas por um segundo canal IPC ou registradas em arquivo de log.
- Como requisitos técnicos obrigatórios, cliente e servidor devem ser executáveis distintos, rodando como processos separados; a comunicação deve usar IPC real o servidor deve usar múltiplas threads; e o acesso à tabela deve ser protegido por mutex ou semáforo.

3 ARQUITETURA E IMPLEMENTAÇÃO

O sistema foi desenvolvido em C++ As threads do servidor usam a biblioteca Pthreads A memória compartilhada e os semáforos entre processos usam os objetos nomeados da API do Windows, que cumprem a mesma função da memória compartilhada POSIX

Arquivo	Função
banco.h	Constantes, nomes dos objetos de IPC, códigos das operações e estruturas compartilhadas (Registro, Requisicao, Resposta e MemoriaCompartilhada).
servidor.c	Cria a memória compartilhada e os objetos de sincronização, carrega o banco do arquivo, cria o pool de 4 threads e processa as requisições.
cliente.c	Conecta-se aos objetos criados pelo servidor, exibe o menu, monta as requisições, insere-as na fila e exibe as respostas.

Arquivo	Função
banco.txt	Persistência dos registros no formato “id;nome”, um por linha.
Makefile	Regras de compilação dos dois executáveis.

3.1 Estruturas de dados compartilhadas

O arquivo banco.h define a estrutura Registro, baseada no exemplo do enunciado e acrescida do campo ocupado, que indica se a posição do vetor está em uso. Define também a requisição enviada pelo cliente, a resposta do servidor e o layout da região de memória compartilhada:

```
typedef struct {
    int id;
    char nome[50];
    int ocupado;
} Registro;

typedef struct {    // enviado pelo cliente
    int op;          // OP_INSERT, OP_SELECT, OP_UPDATE, OP_DELETE, OP_SAIR
    int id;
    char nome[50];
} Requisicao;

typedef struct {    // devolvido pelo servidor
    int status;      // 1 = OK, 0 = Erro
    char msg[100];
    int id;
    char nome[50];
} Resposta;

typedef struct {    // conteúdo da memória compartilhada
    Requisicao fila_req[MAX_FILA]; // fila circular (MAX_FILA = 10)
    int inicio_req;
    int fim_req;
    Resposta resp;
} MemoriaCompartilhada;
```

3.2 Criação da memória compartilhada e dos semáforos

O servidor cria uma região de memória nomeada com CreateFileMappingA e a mapeia em seu espaço de endereçamento com MapViewOfFile. Em seguida, cria os semáforos e mutexes nomeados

```
HANDLE hMap = CreateFileMappingA(INVALID_HANDLE_VALUE, NULL, PAGE_READWRITE, 0,
sizeof(MemoriaCompartilhada), SHM_NOME);
shm = (MemoriaCompartilhada *)MapViewOfFile(hMap, FILE_MAP_ALL_ACCESS, 0, 0,
sizeof(MemoriaCompartilhada));
```

```
// posições livres, itens na fila e trava dos índices
sem_vagas = CreateSemaphoreA(NULL, MAX_FILA, MAX_FILA, SEM_VAGAS);
sem_tarefas = CreateSemaphoreA(NULL, 0, MAX_FILA, SEM_TAREFAS);
mutex_fila = CreateMutexA(NULL, FALSE, MUTEX_FILA);
```

3.3 Cliente como produtor

O cliente aguarda uma posição livre (sem_vagas), trava a fila, grava a requisição na posição fim_req, avança o índice circularmente, destrava a fila e sinaliza sem_tarefas para acordar uma thread do servidor:

```
WaitForSingleObject(sem_vagas, INFINITE); // wait(vagas)
WaitForSingleObject(mutex_fila, INFINITE); // lock(fila)

shm->fila_req[shm->fim_req] = req;
shm->fim_req = (shm->fim_req + 1) % MAX_FILA;

ReleaseMutex(mutex_fila); // unlock(fila)
ReleaseSemaphore(sem_tarefas, 1, NULL); // signal(tarefas)
```

3.4 Pool de threads como consumidor

O servidor cria N_THREADS = 4 threads com pthread_create. Cada uma executa a função trabalhador em laço infinito: espera uma tarefa, retira-a da fila de forma protegida e libera uma vaga para o produtor

```
pthread_t threads[N_THREADS];
for (long i = 0; i < N_THREADS; i++)
    pthread_create(&threads[i], NULL, trabalhador, (void *)i);

void *trabalhador(void *arg) {
    while (1) {
        WaitForSingleObject(sem_tarefas, INFINITE); // wait(tarefas)
        WaitForSingleObject(mutex_fila, INFINITE);
        Requisicao req = shm->fila_req[shm->inicio_req];
        shm->inicio_req = (shm->inicio_req + 1) % MAX_FILA;
        ReleaseMutex(mutex_fila);
        ReleaseSemaphore(sem_vagas, 1, NULL); // signal(vagas)
    }
}
```

3.5 Seção crítica protegida por pthread_mutex_t

A tabela de registros é um vetor global de 100 posições no servidor, compartilhado pelas quatro threads. Todo acesso a ela, tanto de leitura quanto de escrita, ocorre entre pthread_mutex_lock e pthread_mutex_unlock. Um atraso artificial de 300 ms (Sleep) simula a latência de acesso a disco e torna visível, no terminal, a disputa das threads pelo mutex:

```
pthread_mutex_lock(&mutex_sc);
cout << "[Thread " << id_thread << "] entrou na seção crítica." << endl;
Sleep(300); // simula latência de disco
```

```

if (req.op == OP_INSERT) {
    /* procura ID duplicado e primeira posição livre */
    ...
    tabela[livre].id = req.id;
    strcpy(tabela[livre].nome, req.nome);
    tabela[livre].ocupado = 1;
    salvar_banco();          // persiste em banco.txt
} else if (req.op == OP_SELECT) { ... }
else if (req.op == OP_DELETE) { ... }
else if (req.op == OP_UPDATE) { ... }

cout << "[Thread " << id_thread << "] Saiu da Secao Critica." << endl;
pthread_mutex_unlock(&mutex_sc);

```

3.6 Canal de resposta

A resposta volta ao cliente pelo campo resp da memória compartilhada, que funciona como um segundo canal de IPC. Como há um único campo, o acesso é serializado por um mutex nomeado (mutex_resp) e por um protocolo de confirmação com dois semáforos: a thread grava a resposta e sinaliza sem_resp; o cliente lê e sinaliza sem_resp_lida; só então a thread libera o canal para a próxima resposta:

```

// servidor (thread)
WaitForSingleObject(mutex_resp, INFINITE); // um responde por vez
shm->resp = resp;
ReleaseSemaphore(sem_resp, 1, NULL);      // avisa o cliente
WaitForSingleObject(sem_resp_lida, INFINITE); // espera a leitura
ReleaseMutex(mutex_resp);

// cliente
WaitForSingleObject(sem_resp, INFINITE);
Resposta resp = shm->resp;
ReleaseSemaphore(sem_resp_lida, 1, NULL); // confirma a leitura

```

3.7 Persistência

Ao iniciar, o servidor carrega os registros de banco.txt para o vetor em memória. Após cada INSERT, UPDATE ou DELETE bem-sucedido, a função salvar_banco regrava o arquivo inteiro

4 COMPILAÇÃO E EXECUÇÃO

O projeto requer Windows com o compilador g++ do MinGW/MSYS2. A compilação pode ser feita pelo Makefile ou manualmente:

```

make
# ou
g++ -Wall -O2 servidor.c -o servidor.exe -pthread
g++ -Wall -O2 cliente.c -o cliente.exe

```

Para executar, abre-se primeiro o servidor em um terminal (servidor.exe) e, em seguida, o cliente em outro terminal (cliente.exe). O servidor precisa estar rodando antes, pois é ele quem cria os objetos de IPC.

5 RESULTADOS OBTIDOS

A tabela abaixo apresenta o roteiro de testes executado a partir de um cliente, com o banco contendo inicialmente apenas o registro 1;hugo.

#	Operação	Entrada	Resposta do servidor
1	SELECT	id = 1	SUCESSO – Encontrado (Nome = hugo)
2	INSERT	id = 2, nome = maria	SUCESSO – Inserido com sucesso
3	INSERT	id = 2, nome = joao	ERRO – ID ja existe
4	UPDATE	id = 2, nome = ana	SUCESSO – Atualizado com sucesso
5	SELECT	id = 2	SUCESSO – Encontrado (Nome = ana)
6	DELETE	id = 2	SUCESSO – Removido com sucesso
7	SELECT	id = 2	ERRO – ID nao encontrado
8	UPDATE / DELETE	id = 99	ERRO – ID nao encontrado
9	Reiniciar servidor	—	Registros de banco.txt recarregados

```
=====
CLIENTE DO BANCO
=====
```

- [1] Inserir (INSERT)
- [2] Consultar (SELECT)
- [3] Atualizar (UPDATE)
- [4] Deletar (DELETE)
- [9] Desligar Servidor (SAIR)
- [0] Fechar Cliente

```
=====
Escolha uma opcao: 2
Digite o ID: 1
```

```
>>> RESPOSTA DO SERVIDOR <<<
Status: [SUCESSO]
Mensagem: Encontrado
Dados: ID = 1 | Nome = hugo
-----
```

```
=====
CLIENTE DO BANCO
=====
```

- [1] Inserir (INSERT)
- [2] Consultar (SELECT)
- [3] Atualizar (UPDATE)
- [4] Deletar (DELETE)
- [9] Desligar Servidor (SAIR)
- [0] Fechar Cliente

```
=====
Escolha uma opcao: 1
Digite o ID: 2
Digite o Nome: maria
```

```
>>> RESPOSTA DO SERVIDOR <<<
Status: [SUCESSO]
```

```
=====
          CLIENTE DO BANCO
=====
```

- [1] Inserir (INSERT)
- [2] Consultar (SELECT)
- [3] Atualizar (UPDATE)
- [4] Deletar (DELETE)
- [9] Desligar Servidor (SAIR)
- [0] Fechar Cliente

```
=====
```

Escolha uma opcao: 1

Digite o ID: 2

Digite o Nome: joao

>>> RESPOSTA DO SERVIDOR <<<

Status: [ERRO]

Mensagem: Erro: ID ja existe

```
-----
```

```
=====
          CLIENTE DO BANCO
=====
```

- [1] Inserir (INSERT)
- [2] Consultar (SELECT)
- [3] Atualizar (UPDATE)
- [4] Deletar (DELETE)
- [9] Desligar Servidor (SAIR)
- [0] Fechar Cliente

```
=====
```

Escolha uma opcao: 3

Digite o ID: 2

Digite o Novo Nome: ana

```
=====
          CLIENTE DO BANCO
=====
```

- [1] Inserir (INSERT)
- [2] Consultar (SELECT)
- [3] Atualizar (UPDATE)
- [4] Deletar (DELETE)
- [9] Desligar Servidor (SAIR)
- [0] Fechar Cliente

```
=====
Escolha uma opcao: 2
Digite o ID: 2
```

```
>>> RESPOSTA DO SERVIDOR <<<
Status: [SUCESSO]
Mensagem: Encontrado
Dados: ID = 2 | Nome = ana
-----
```

```
=====
          CLIENTE DO BANCO
=====
```

- [1] Inserir (INSERT)
- [2] Consultar (SELECT)
- [3] Atualizar (UPDATE)
- [4] Deletar (DELETE)
- [9] Desligar Servidor (SAIR)
- [0] Fechar Cliente

```
=====
Escolha uma opcao: 4
Digite o ID: 2
```

```
>>> RESPOSTA DO SERVIDOR <<<
```

```
>>> RESPOSTA DO SERVIDOR <<<
Status: [SUCESSO]
Mensagem: Removido com sucesso
-----
```

```
=====
CLIENTE DO BANCO
=====
```

```
[1] Inserir (INSERT)
[2] Consultar (SELECT)
[3] Atualizar (UPDATE)
[4] Deletar (DELETE)
[9] Desligar Servidor (SAIR)
[0] Fechar Cliente
=====
```

```
Escolha uma opcao: 2
Digite o ID: 2
```

```
>>> RESPOSTA DO SERVIDOR <<<
Status: [ERRO]
Mensagem: Erro: ID nao encontrado
-----
```

```
=====
CLIENTE DO BANCO
=====
```

```
[1] Inserir (INSERT)
[2] Consultar (SELECT)
[3] Atualizar (UPDATE)
[4] Deletar (DELETE)
[9] Desligar Servidor (SAIR)
[0] Fechar Cliente
=====
```

```
Escolha uma opcao: 0
[CLIENTE] Fechando cliente (Servidor continua rodando)...
```

```
[SERVIDOR] Pool com 4 threads criado com sucesso!  
[SERVIDOR] Aguardando clientes enviarem tarefas...
```

```
[Thread 2] Recebeu requisicao OP: 2, ID: 1  
[Thread 2] entrou na seção crítica.
```

```
=== BANCO DE DADOS ===  
ID: 1 | Nome: hugo  
=====
```

```
[Thread 2] Saiu da Secao Critica.  
[Thread 1] Recebeu requisicao OP: 1, ID: 2  
[Thread 1] entrou na seção crítica.
```

```
=== BANCO DE DADOS ===  
ID: 1 | Nome: hugo  
ID: 2 | Nome: maria  
=====
```

```
[Thread 1] Saiu da Secao Critica.  
[Thread 3] Recebeu requisicao OP: 1, ID: 2  
[Thread 3] entrou na seção crítica.
```

```
=== BANCO DE DADOS ===  
ID: 1 | Nome: hugo  
ID: 2 | Nome: maria  
=====
```

```
[Thread 3] Saiu da Secao Critica.  
[Thread 4] Recebeu requisicao OP: 3, ID: 2  
[Thread 4] entrou na seção crítica.
```

```

=== BANCO DE DADOS ===
ID: 1 | Nome: hugo
ID: 2 | Nome: ana
=====

[Thread 4] Saiu da Secao Critica.
[Thread 2] Recebeu requisicao OP: 2, ID: 2
[Thread 2] entrou na seção crítica.

=== BANCO DE DADOS ===
ID: 1 | Nome: hugo
ID: 2 | Nome: ana
=====

[Thread 2] Saiu da Secao Critica.
[Thread 1] Recebeu requisicao OP: 4, ID: 2
[Thread 1] entrou na seção crítica.

=== BANCO DE DADOS ===
ID: 1 | Nome: hugo
=====

[Thread 1] Saiu da Secao Critica.
[Thread 3] Recebeu requisicao OP: 2, ID: 2
[Thread 3] entrou na seção crítica.

=== BANCO DE DADOS ===
ID: 1 | Nome: hugo
=====

[Thread 3] Saiu da Secao Critica.

```

6 ANÁLISE E DISCUSSÃO

Os resultados mostram que o sistema cumpre os requisitos obrigatórios da proposta. Cliente e servidor são executáveis distintos que rodam como processos separados; a comunicação usa memória compartilhada real do sistema operacional, sem variáveis globais nem polling de arquivos; o servidor mantém um pool de quatro threads; e a tabela compartilhada é protegida por pthread_mutex_t. A fila circular sincronizada por semáforos implementa corretamente o padrão produtor-consumidor: o cliente bloqueia quando a fila está cheia e as threads bloqueiam quando ela está vazia, sem espera ativa.

A análise também revelou limitações que devem ser discutidas:

- **Paralelismo limitado pela granularidade do mutex.** Como a operação inteira e o atraso simulado ficam dentro de um único mutex, as threads processam as requisições uma de cada vez, e o ganho real está apenas em receber e enfileirar pedidos enquanto outro é atendido.

- **Canal de resposta único.** Há um só campo de resposta na memória compartilhada. Com um cliente, o protocolo funciona corretamente. Com vários clientes simultâneos, porém, um cliente pode ler a resposta destinada a outro, pois não há identificação do remetente. A solução seria incluir um identificador de cliente na requisição e manter um espaço de resposta por cliente.
- **Robustez.** O código não verifica o retorno de todas as chamadas de criação de recursos e não limita o tamanho do nome digitado no cliente. Nomes com espaços também não são aceitos pelo menu.

7 CONCLUSÃO

O trabalho permitiu aplicar na prática os conceitos de processos, threads, comunicação entre processos e exclusão mútua. A implementação demonstra um cliente e um servidor independentes trocando dados por memória compartilhada, uma fila produtor-consumidor sincronizada por semáforos e um pool de threads que acessa uma tabela protegida por mutex, com persistência em arquivo.

Os testes confirmaram o funcionamento das quatro operações e a persistência dos dados após reiniciar o servidor. A análise mostrou que a principal oportunidade de evolução é aumentar o paralelismo efetivo, reduzindo o tamanho da seção crítica, e tornar o canal de respostas seguro para múltiplos clientes.